Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Machine Learning — Midterm Project — October 2026

Team leader: [Team leader name] **Members:** [Member names, or 'individual project']

Code: https://github.com/[your-username]/who-wrote-it

Abstract. We build a balanced, fully parallel dataset of 4,824 (LLM, prompt, response) triples: the same 804 AlpacaEval instructions answered by six LLM families (GPT-4o, Claude 3 Opus, Gemini Pro, Llama 3 70B, Qwen 1.5 72B, DeepSeek 67B). From the response text alone, a TextCNN reaches 0.577 ± 0.012 test accuracy and a BiLSTM 0.576 ± 0.011 (chance = 0.167; mean ± std over 3 seeds; test prompts never seen in training). The prompt alone carries no signal in a parallel design (exactly 1/6 accuracy, by construction), and adding it to the response changes accuracy by only +0.022 (CNN) and +0.000 (LSTM); we show that input-only accuracy only rises above chance when the collection is confounded (topic-biased model choice). For extra credit we test generalization across task domains, prompt sources and unseen model versions (RQ3), and explain the fingerprints with stylometric features, n-grams, CNN filters and six controlled text ablations (RQ4): the fingerprint is mostly stylistic. Replacing every content word with a placeholder leaves CNN accuracy at 0.589 (full text: 0.577), content words alone give only 0.240, and stripping markdown and line breaks costs the most (0.428).

1. Problem definition

Given a text y produced by an LLM in reply to a user prompt x, predict the family $c \in$ {GPT, Claude, Gemini, Llama, Qwen, DeepSeek} of the model that wrote it. This is a 6-way, single-label text classification problem. We study three input views: the **output** y (RQ1), the **input** x alone, and the **pair** (x, y) (RQ2). We report accuracy and macro-F1 (classes are exactly balanced, so the two agree closely; chance accuracy is 1/6 = 0.167). A useful detector must generalise to *new prompts*, so every split is made at the prompt level: all six answers to a prompt are in the same split, and no test prompt is seen during training.

2. Dataset curation

Source. We use the public model-output files of AlpacaEval (Li et al., 2023; github.com/tatsu-lab/alpaca_eval, Apache-2.0 licence; commit cd543a1). AlpacaEval asks every evaluated model the same 805 instructions and publishes each model's raw answer in `results/<model>/model_outputs.json` with fields `instruction`, `output`, `generator` and `dataset` (the prompt's origin). This gives exactly the required triple (LLM_name = generator, LLM_input = instruction, LLM_output = output). The prompts are public benchmark prompts (Self-Instruct, OpenAssistant, Koala, Anthropic Helpful-Base, Vicuna), so the data contain no private or personal information, and the outputs are redistributed by the benchmark authors under a permissive licence. `src/download_data.sh` re-downloads the files from the pinned commit.

Construction. We picked one large flagship chat model per family, all released between late 2023 and May 2024, giving six families — more than the required three. Files are aligned by instruction text (the Gemini file is stored in a different order, which we caught with an assertion). One prompt (id 104) has an empty Gemini answer, so we drop that prompt for every model to keep the classes exactly balanced: **804 prompts × 6 families = 4,824 examples**. Four families also have a sibling version in AlpacaEval, which we hold out entirely for the cross-version test in RQ3.

Table 1. The six LLM families (804 responses each). Medians are computed on full responses.

Family	Model used for training/testing	Held-out sibling (RQ3 only)	Median words	Median chars
GPT	gpt-4o-2024-05-13	gpt-4-turbo-2024-04-09	273	1858
Claude	claude-3-opus-20240229	claude-3-5-sonnet-20240620	204	1310
Gemini	gemini-pro	—	223	1410
Llama	Meta-Llama-3-70B-Instruct	Meta-Llama-3.1-70B-Instruct-Turbo	298	1940
Qwen	Qwen1.5-72B-Chat	Qwen2-72B-Instruct	244	1534
DeepSeek	deepseek-llm-67b-chat	—	174	1065

Splits. Prompts are shuffled with a fixed seed and split 70/15/15: 562/120/122 prompts, i.e. 3,372/720/732 examples. The validation set is used only for early stopping. **Prompt sources:** selfinstruct 252, oasst 188, koala 156, helpful_base 128, vicuna 80. **Task domains** (assigned by transparent keyword rules in `prepare_data.py`, used in RQ3): qa 606, writing 100, coding 74, math 24 prompts. The median prompt has 18 words; responses are far longer and their lengths differ by family (Figure 1).

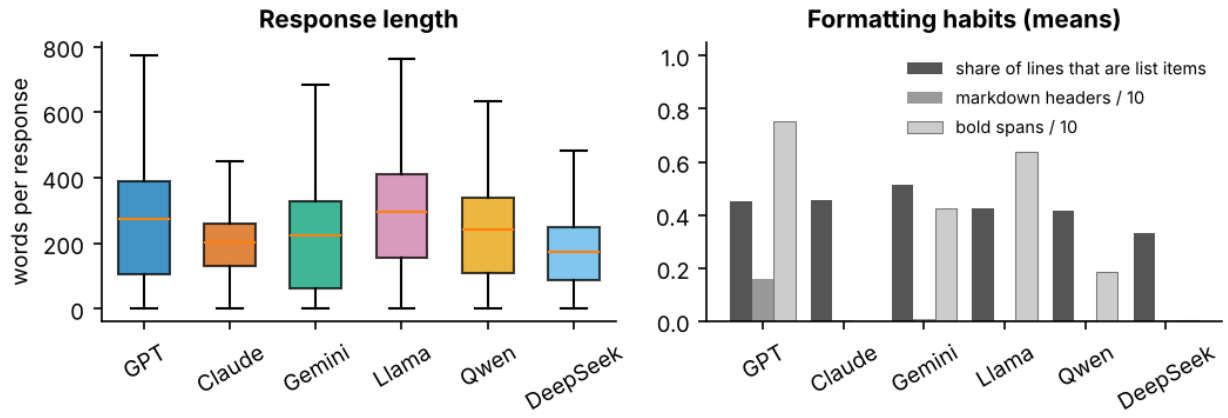


Figure 1. Left: response length by family (box = interquartile range, whiskers = 1.5 IQR, outliers hidden). Right: average formatting habits. Even before any learning, families differ visibly in length and markdown use.

3. Models and training

Tokenisation. A regular-expression tokenizer splits text into words, numbers and *individual punctuation characters*, keeps case, and turns each line break into a `<nl>` token. This keeps markdown such as `**`, ###, `-` and list numbering visible to the models, which matters because formatting may be part of a fingerprint. The vocabulary is built on the training split only (tokens with frequency ≥ 2; 18,430 types for the output view). Responses are truncated to their first 256 tokens (about 1,150 characters). Median response lengths range from 232 tokens (DeepSeek) to 440 (Llama), so between 42% and 73% of a family's responses are cut; this cap keeps the BiLSTM affordable on a CPU, and the length of the visible part still differs by family. In the input+output view the prompt is truncated to 96 tokens and joined to the response with a `<sep>` token. We checked the effect of the 256-token limit by re-training the CNN with 512 tokens (Section 4).

CNN (Kim, 2014). Embedding (128-d, learned from scratch) → four parallel 1-D convolutions with widths 2, 3, 4, 5 and 100 filters each (ReLU) → max-over-time pooling (padding positions masked) → dropout 0.5 → linear layer to 6 logits. 2,541,046 parameters. Each filter is an n-gram detector, which later lets us read off what it responds to (RQ4).

RNN. Embedding (128-d) with dropout 0.2 → one-layer *bidirectional* LSTM (96 units per direction) run on packed sequences so padding is never read → concatenation of max-pooling and mean-pooling over valid time steps (384-d) → dropout 0.4 → linear layer. 2,534,918 parameters. Pooling over all states (rather than using only the last state) lets the model use cues from anywhere in the response.

Table 2. Training details. Seeds 0, 1, 2 for the main experiments; batches are formed from length-sorted chunks of 20 batches (then shuffled) to limit padding.

	CNN	BiLSTM
Optimiser	AdamW, lr 1e-3, weight decay 1e-4	AdamW, lr 2e-3, weight decay 1e-4
Batch size / max epochs	32 / 15	32 / 12
Early stopping	validation macro-F1, patience 3, restore best epoch	same
Regularisation	dropout 0.5, gradient clipping 1.0	embedding dropout 0.2, dropout 0.4, clipping 1.0
Loss	cross-entropy (classes balanced)	same
Epochs actually run (3 seeds)	8, 15, 12	9, 12, 10
Time per epoch / per run (1 CPU core)	17 s / 4.2 min	35 s / 7.0 min

Reference baseline. To put the neural results in context we also train a TF-IDF (word 1–2-grams on the same tokens, sublinear tf, up to 200k features) + multinomial logistic regression classifier (C = 4) on train+validation. All code runs on a 2-core CPU; the whole pipeline (data → all experiments → figures → this PDF) is scripted and described in the README.

4. RQ1 — Can we identify which LLM generated a response?

Table 3. RQ1 test results on 732 held-out responses (122 unseen prompts × 6). Neural rows: mean ± std over 3 seeds; per-family columns are F1 scores.

Model (output only)	Accuracy	Macro-F1	GPT	Claude	Gemini	Llama	Qwen	DeepSeek
TextCNN	0.577 ± 0.012	0.586 ± 0.011	0.65	0.60	0.66	0.70	0.39	0.52
BiLSTM	0.576 ± 0.011	0.575 ± 0.012	0.67	0.59	0.66	0.69	0.39	0.46
TF-IDF + LogReg	0.620	0.617	0.72	0.66	0.67	0.72	0.38	0.56
Chance	0.167	0.167	0.17	0.17	0.17	0.17	0.17	0.17

Answer: yes. Both required models identify the author family far above chance: the CNN reaches 0.577 ± 0.012 accuracy and the BiLSTM 0.576 ± 0.011, i.e. roughly 3.5× and 3.5× chance, on prompts neither model saw in training. Seed-to-seed variation is small (std ≤ 0.012), so the ranking is not a fluke of initialisation. The simple TF-IDF baseline is slightly better than both networks (0.620), which already hints that much of the signal is in *which* words and symbols occur rather than in long-range order (RQ4 tests this directly).

The easiest family for the CNN is **Llama** (F1 0.70) and the hardest is **Qwen** (F1 0.39); for the BiLSTM the easiest is Llama (0.69) and the hardest Qwen (0.39). The most frequent CNN errors are DeepSeek→Qwen (26.2% of DeepSeek responses), Qwen→Claude (23.5%) and GPT→Qwen (21.0%); the BiLSTM's top confusion is DeepSeek→Qwen (21.9%). Errors are not spread uniformly (Figure 2). Qwen is the most common wrong guess (37% of all CNN errors), and it is also the hardest family to recognise: its answers have the least distinctive style (its stylometric profile in Figure 8 is the closest to the average of all families), so responses without strong cues tend to be assigned to it. The easiest families have strong, consistent templates, such as Llama's 'Here is/are…' openers and Gemini's '*'-bulleted layout (Tables 9–11).

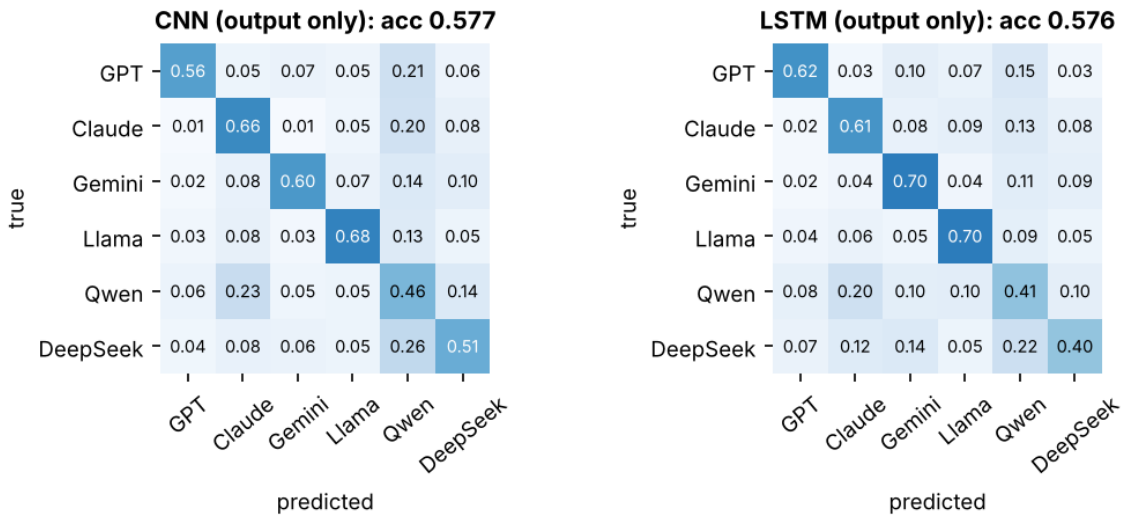


Figure 2. Row-normalised confusion matrices on the test set, summed over the three seeds.

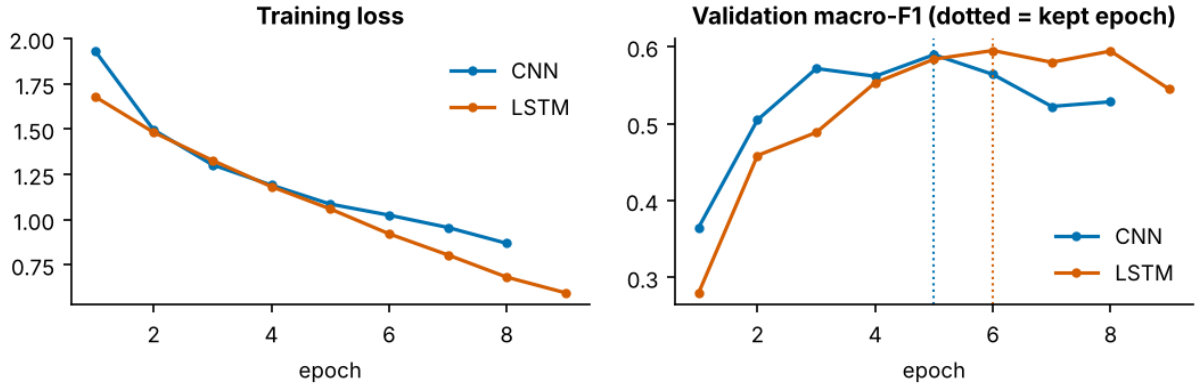


Figure 3. Training loss and validation macro-F1 per epoch (seed 0). Both models start to over-fit after a few epochs (training loss keeps falling while validation F1 plateaus), which is why early stopping restores the best epoch.

Truncation check. Re-training the CNN (seed 0) on the first 512 instead of 256 tokens gives 0.574 accuracy versus 0.583 for the same seed with 256 tokens, so the 256-token budget used for both networks costs little; the beginning of a response already carries most of the fingerprint (see also the 64-token ablation in RQ4).

5. RQ2 — Does the user's prompt help identify the LLM?

Table 4. RQ2 on the main (parallel) dataset. Neural models: mean ± std over seeds (number of seeds for input / output / input+output: CNN 3/3/3, BiLSTM 3/3/3; a single value means one seed).

Input view	TextCNN acc.	BiLSTM acc.	TF-IDF+LR acc.	TextCNN macro-F1	BiLSTM macro-F1
Input only	0.167 ± 0.001	0.167 ± 0.000	0.167	0.120 ± 0.022	0.094 ± 0.010
Output only	0.577 ± 0.012	0.576 ± 0.011	0.620	0.586 ± 0.011	0.575 ± 0.012
Input + Output	0.600 ± 0.011	0.577 ± 0.012	0.608	0.613 ± 0.006	0.580 ± 0.009

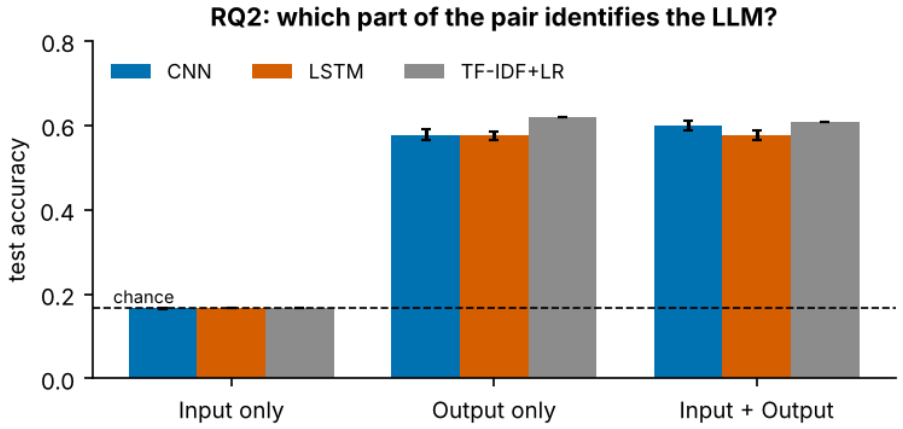


Figure 4. RQ2: accuracy of the three input views (error bars = std over seeds; dashed line = chance).

(i) Can the LLM be predicted from the prompt alone? Not in our data — and provably so. Every prompt appears six times, once with each label. A classifier that sees only the prompt must give the same prediction to all six copies, so exactly one of them is right: accuracy is 1/6 for *any* model, however well trained. The measured input-only accuracies (0.167 CNN, 0.167 LSTM, 0.167 TF-IDF) confirm this; the tiny deviations from 0.1667 come from a handful of prompts whose six identical copies landed in differently sized evaluation batches, so floating-point round-off broke a near-tie between two classes differently (we found 2 such prompts out of 122 for one seed). Macro-F1 is even lower because the models collapse onto a few classes. This is a useful sanity check of the pipeline: a bug that leaked labels into the input would have shown up here.

(ii) Does adding the prompt help? Only slightly, and only for the CNN. Input + output changes accuracy by +0.022 for the CNN and +0.000 for the BiLSTM relative to output only (-0.012 for TF-IDF); the largest seed standard deviation in these four settings is 0.012. Both views see the same first 256 response tokens, so the comparison isolates the prompt. Because the prompt carries no label information by itself in a parallel dataset, any effect must come from *interaction*: knowing what was asked lets a model judge the answer relative to the request (a bulleted list in reply to a yes/no question, a long essay in reply to a one-line question). The small gain for the CNN suggests it picks up a little of this context, but the effect is minor compared with the information in the response itself, and the identical prompt tokens shared by all six classes are noise from the classifier's point of view.

(iii) When would input-only classification work surprisingly well? When the data collection is confounded. To show this we built two non-parallel versions of the dataset, keeping a single response per prompt. In the *fair* version the responding family is drawn uniformly at random. In the *biased* version we cluster the prompts into six equal-size topic groups (k-means on TF-IDF of the instruction, capacity-constrained) and each group 'prefers' one family with probability 0.7 — mimicking real chat logs where, say, programmers favour one assistant and novelists another. Labels stay balanced in both versions.

Table 5. RQ2 confound experiment (one response per prompt; 562 training prompts, same prompt split; seed 0).

Collection protocol	Majority class	Input only (CNN / TF-IDF)	Output only (CNN / TF-IDF)	Input + Output (CNN / TF-IDF)
Fair (random family)	0.221	0.189 / 0.148	0.508 / 0.484	0.377 / 0.475
Biased (topic → family, p = 0.7)	0.230	0.238 / 0.303	0.467 / 0.500	0.467 / 0.525

With fair collection, the prompt is useless: input-only accuracy is 0.189 (CNN) / 0.148 (TF-IDF), no better than predicting the majority class (0.221 on this small test set). With biased collection the same TF-IDF classifier predicts the 'author' from the prompt alone at 0.303, clearly above the 0.230 majority baseline, without ever seeing a response; the CNN, which has to learn the topic→family mapping from only 562 short prompts, reaches 0.238. Nothing about the LLMs changed — the classifier learned *who tends to ask what*. The lesson for RQ2 is that a high input-only score is a symptom of selection bias (users, platforms, time periods or task mixes that differ by model), not evidence of a

model fingerprint, and it inflates input+output and even output-only scores in the same way. Parallel prompts, as in our main dataset, remove the confound by design.

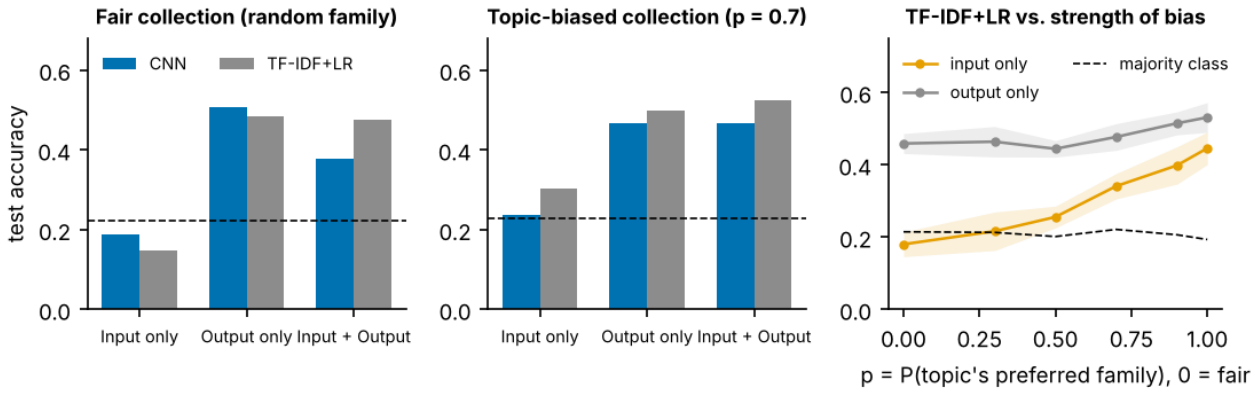


Figure 5. Left, middle: the three input views under fair and topic-biased collection (p = 0.7; dashed line = majority class). Right: TF-IDF accuracy as the bias grows (mean ± std over 5 random assignments). Input-only accuracy rises above chance only when the choice of model depends on the prompt topic.

6. RQ3 (extra credit) — Do LLM fingerprints generalise across tasks and domains?

We test three kinds of shift. All use the output-only view and the same architectures and hyper-parameters as RQ1.

(a) Unseen task domain. Experiment W trains on technical (code/math) + general QA prompts and tests on *writing* prompts (emails, stories, posts…). Experiment T trains on general QA and tests on *code/math*. To isolate the effect of the domain from the effect of training-set size, we compare against an *in-domain* control with the same test set and the same number of training prompts: the target prompts are split in two halves, the test half is fixed, and the control swaps an equal number of source prompts for the other target half. We report the mean over the two folds (each half serves once as the test set).

Table 6. RQ3a cross-task generalisation (mean ± std over the 2 folds; one seed per fold).

Experiment	Model	In-domain control	Unseen domain	Change
W: test = writing	TextCNN	0.497 ± 0.007	0.425 ± 0.002	-0.072
W: test = writing	BiLSTM	0.460 ± 0.007	0.477 ± 0.020	+0.017
W: test = writing	TF-IDF+LR	0.530 ± 0.010	0.505 ± 0.002	-0.025
T: test = code/math	TextCNN	0.485 ± 0.036	0.480 ± 0.020	-0.005
T: test = code/math	BiLSTM	0.534 ± 0.061	0.468 ± 0.029	-0.066
T: test = code/math	TF-IDF+LR	0.561 ± 0.034	0.537 ± 0.044	-0.024

Removing the target domain from training changes accuracy by between -0.072 and +0.017 across the 6 model/experiment combinations (average -0.029). Performance on an unseen task decreases, but only moderately; the classifiers nevertheless stay far above chance on writing and on code/math they never trained on. So a large part of what they learn is a *task-independent* house style (how answers open, how they are laid out, typical phrasing), with a smaller task-specific part (e.g. how each family formats code blocks or a cover letter) that is lost under shift. Note that the test sets here are small (about 300 responses per fold), so differences of a few points are within noise.

(b) Unseen prompt source. We hold out each of the five prompt collections in turn (leave-one-source-out) and train on the other four. The collections differ strongly in style: Self-Instruct prompts are short synthetic tasks, Koala/OASST are real user requests, Helpful-Base comes from Anthropic's HH data.

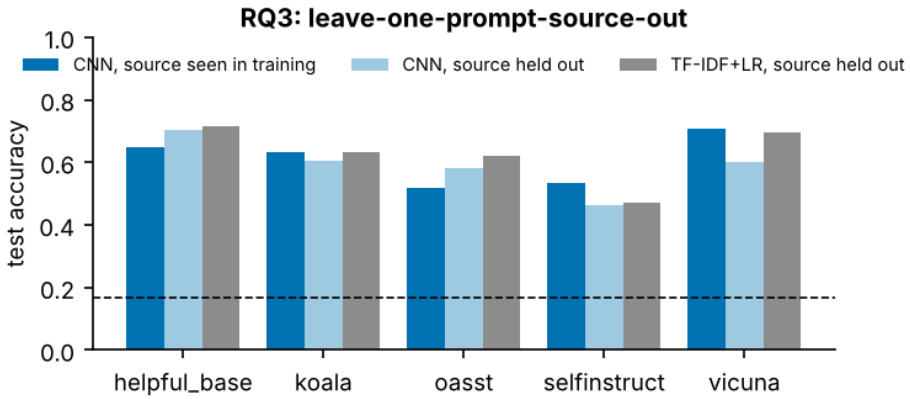


Figure 6. Leave-one-source-out accuracy versus the RQ1 CNN's accuracy on test prompts from the same source (which it saw other prompts of during training).

Held-out-source accuracy for the CNN ranges from 0.462 to 0.704 (TF-IDF: 0.472–0.714); on average it is 0.017 below the in-distribution accuracy on the same source. The hardest source is **selfinstruct** and the easiest are helpful_base, vicuna, koala. Self-Instruct's short, closed tasks ('classify this sentence', 'give a synonym') leave little room for stylistic freedom: when an answer is one line, there are few places for a fingerprint to show. Identifiability therefore depends more on *how much freedom the prompt gives the model* than on whether that prompt source was seen in training.

(c) Unseen model version. A practical detector will meet new releases. We take the RQ1 models (trained on one version per family) and test them, without retraining, on the answers of a sibling model from the same lab to the same test prompts.

Table 7. RQ3c cross-version generalisation (recall = share of that family's responses attributed to the right family; mean over 3 seeds).

Family	Trained on	Tested on (never seen)	CNN recall: same version	CNN recall: sibling	LSTM recall: same	LSTM recall: sibling
GPT	gpt-4o-2024-05-13	gpt-4-turbo-2024-04-09	0.563	0.522	0.623	0.549
Claude	claude-3-opus-20240229	claude-3-5-sonnet-20240620	0.656	0.751	0.612	0.716
Llama	Meta-Llama-3-70B-Instruct	Meta-Llama-3.1-70B-Instruct-Turbo	0.680	0.658	0.705	0.691
Qwen	Qwen1.5-72B-Chat	Qwen2-72B-Instruct	0.464	0.361	0.410	0.314

Over these four families, CNN accuracy goes from 0.591 on the training versions to 0.573 on the unseen siblings (-0.018); BiLSTM: 0.587 → 0.568 (-0.020). **The fingerprint largely transfers to new versions of the same family.** Claude is recognised even better in the sibling version than in the version the model was trained on, which suggests that a lab's house style can become *more* pronounced from one release to the next. The largest drop is for Qwen (0.46 → 0.36). Most frequent mistakes on the siblings (CNN, pooled over seeds): GPT → Qwen (24%); Claude → Qwen (13%); Llama → Qwen (11%); Qwen → GPT (22%). So the classifier is learning something closer to a *lab-level* style (shared post-training data, templates and formatting conventions) than a fingerprint of one checkpoint, although a deployed detector would still benefit from training on several versions per family.

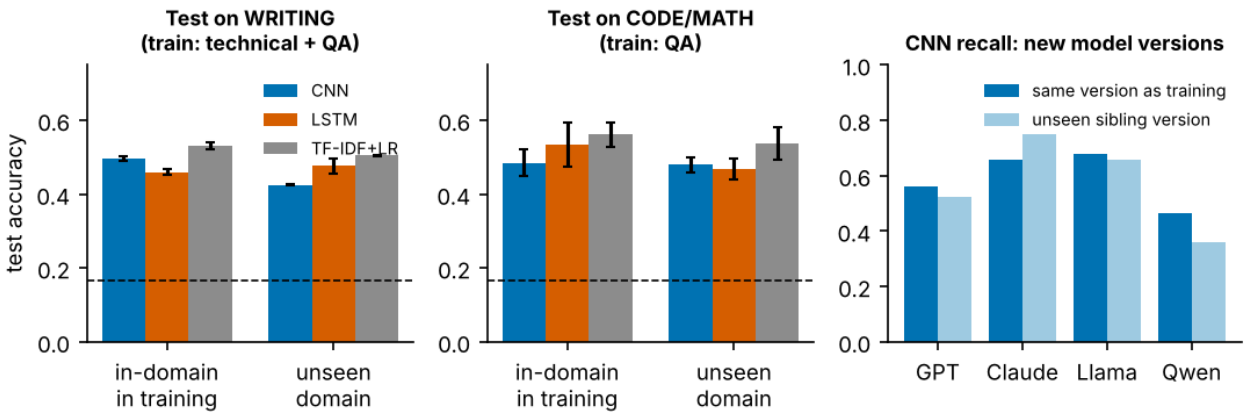


Figure 7. RQ3 summary: cross-task (left, middle; dashed = chance) and cross-version (right) generalisation.

7. RQ4 (extra credit) — What characteristics distinguish the LLMs?

We attack the 'why' from four directions: (a) interpretable stylometric features, (b) the n-grams that the linear model and the CNN rely on, (c) how answers begin, and (d) controlled ablations that remove one kind of signal from the text and retrain the networks.

(a) Stylometry. We compute 29 hand-crafted features on the full response, in five groups: *length* (characters, words, lines, paragraphs), *formatting* (markdown headers, bullets, numbered items, bold spans, code blocks, share of list lines, lines ending in a colon), *lexical* (mean word and sentence length, type-token ratio on the first 200 words, rates of 'I', 'you' and hedging words), *punctuation* (rates of ! ? : ; dashes, parentheses, emoji) and *discourse* (opens with an affirmation such as 'Certainly', opens by restating, has a conclusion phrase, offers further help, ends with a question). Figure 8 shows each family's profile.

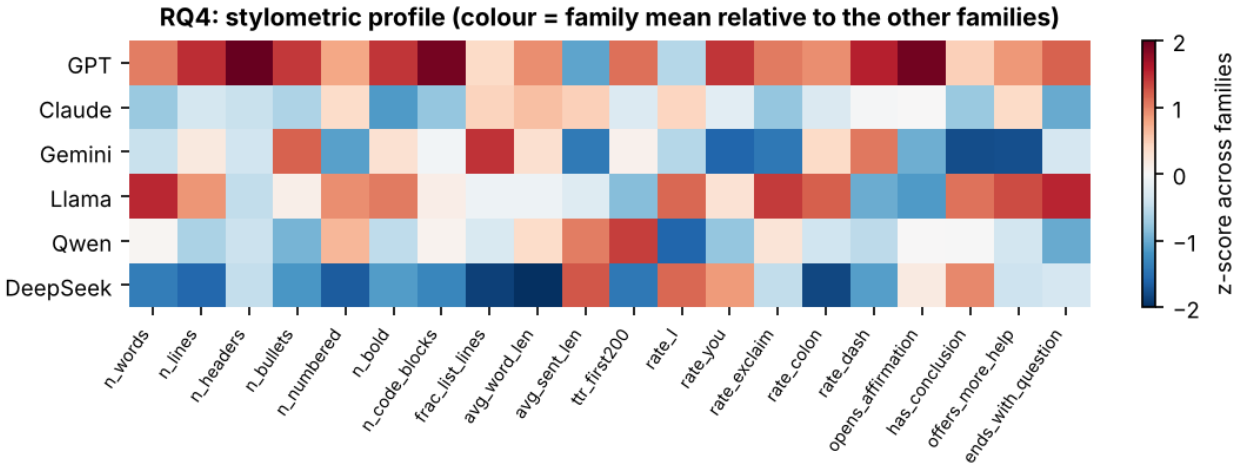


Figure 8. Family means of selected stylometric features, shown as z-scores across the six families: red = this family uses the feature more than the others, blue = less. Raw means are in results/summary.json.

The profiles are distinct. Llama writes the longest answers and DeepSeek the shortest (median 298 vs 174 words). GPT uses the most markdown headers and the most bold text, Gemini the highest share of list lines, and DeepSeek the lowest. GPT most often opens with an affirmation (29% of answers), Llama most often offers further help (9%), and Qwen has the highest lexical diversity. These are exactly the kinds of cues a reader notices, and they are learnable.

Table 8. Classifying with stylometric features only (same split as RQ1).

Feature set (gradient-boosted trees)	Test accuracy	Feature set	Test accuracy
All features	0.590	All (logistic regression)	0.482
Only length	0.358	All except length	0.544
Only formatting	0.430	All except formatting	0.518
Only lexical	0.276	All except lexical	0.600
Only punctuation	0.321	All except punctuation	0.557
Only discourse	0.306	All except discourse	0.557

With only these 29 numbers, gradient-boosted trees reach 0.590 accuracy — on par with the networks that read the actual text. The most informative single group is **formatting** (0.430 alone), and removing **formatting** hurts most (0.590 → 0.518). Permutation importance on the test set ranks `n_bold` (0.130), `n_colon_lines` (0.048), `n_paragraphs` (0.047), `ttr_first200` (0.042), `opens_affirmation` (0.041), `n_headers` (0.031) highest (accuracy drop when the feature is shuffled). So the signals are **both structural and linguistic**: length and layout carry a lot, but lexical and discourse habits add information that layout alone does not.

(b) Discriminative n-grams. The largest-weight n-grams of the TF-IDF logistic regression (Table 9) and the n-grams that maximally activate the CNN filters with the highest weight for each class (Table 10) point to the same kinds of cues: answer openers and sign-offs, markdown idioms, and pet words. Some are surprisingly low-level: GPT-4o's top feature is the typographic apostrophe (’ instead of '), which appears in 22% of its answers versus <1% for every other family. Qwen's symbol list contains the Chinese character U+7684 (‘de’): 5.8% of Qwen answers slip into Chinese characters, versus <1% for any other family. Claude's list is led by 'I apologize', the opening of its refusals and 'I cannot do that' answers (4% of Claude answers, <1% elsewhere); Llama's includes 'What a … question!' (7% of Llama answers open this way, <1% elsewhere); DeepSeek's includes 'Step' and 'It is important to'. (\n = line break.)

Table 9. Most class-indicative n-grams (largest logistic-regression coefficients, among the top 60 per family). Tokens are separated by spaces.

Family	Top word n-grams	Top symbol-only n-grams
GPT	`[U+2019] s`, `Sure`, `s`, `! Here`, `Sure !`, `. Here`, `Here`, `Certainly !`, `Certainly`, `Sure ,`	`# #`, `* *`, `#`, `- *`
Claude	`I apologize`, `apologize`, `apologize ,`, `the`, `or`, `on`, `your`, `Here's`, `Here`, `to`	`\n \n`, `: \n`, `. \n`, `\n 1`
Gemini	`* The`, `- The`, `Therefore ,`, `Therefore`, `Alps`, `It is`, `* This`, `you are`, `that is`, `cannot`	`: *`, `*`, `\n -`, `* *`
Llama	`Here is`, `Here`, `What`, `What a`, `question`, `, with`, `question !`, `Here's`, `NOT`, `* The`	`\n *`, `*`, `* *`, `* :`
Qwen	`but`, `or`, `might`, `, but`, `like`, `Category :`, `Category`, `, and`, `. Here's`, `Remember ,`	`* :`, `)`, `(`, `[U+7684]`
DeepSeek	`\n Step`, `is`, `and`, `It is`, `important`, `also`, `Overall ,`, `important to`, `Overall`, `that`	`* 1`, `=`, `.`

Table 10. What the trained CNN (seed 0) detects. Each entry is one filter.

Family	Most frequent max-activating n-grams of that class's highest-weight CNN filters (count over test answers)
GPT	`e . g . ,` (4); `\n # #` (34); `# # # 2` (6); `# # # 2` (6); `Certainly ! Below is a` (4); `# # # 2 .` (3)
Claude	`and problem - solving skills` (2); `Practice regularly : Consistent practice` (2); `\n \n 5 .` (36); `. \n \n 4 .` (39); `The Hitchhiker's Guide to the` (2); `_ _ Here is a` (7)
Gemini	`Instructions : * * \n` (2); `1 teaspoon baking` (2); `\n \n * *` (17); `\n - *` (3); `_ In the realm of` (2); `variety of` (2)
Llama	`* : This` (3); `: \n \n * *` (38); `Here are` (36); `question !` (6); `_ Here is a` (15); `some examples : \n` (3)
Qwen	`This ensures` (2); `6 . 1 /` (2); `numerous Broadway shows , such` (1); `\n Please note` (2); `, here's a Python` (2); `a step - by` (3)
DeepSeek	`be happy to help you` (2); `that can be` (4); `: < https : /` (2); `` ` ` ` python \n `` (2); `, you can _ _` (2); `you can` (11)

Table 11. How each family begins its answers.

Family	Most common first two words of an answer (share of that family's answers)
GPT	'Certainly Here' 3%; 'Sure Here' 2%; 'Of course' 2%; 'Sure here' 2%; 'Certainly Below' 2%
Claude	'Here are' 6%; 'Here is' 6%; 'I apologize' 4%; 'There are' 4%; 'Here's a' 4%
Gemini	'In the' 2%; 'It is' 1%; 'Ingredients cup' 1%; (starts with a symbol or code) 1%; 'python def' 1%
Llama	'What a' 13%; 'Here is' 11%; 'Here are' 9%; 'Here's a' 6%; 'What an' 2%
Qwen	'There are' 4%; 'Of course' 3%; 'As an' 2%; 'Sure here's' 2%; 'In the' 1%
DeepSeek	'There are' 5%; 'As an' 3%; 'Sure here' 2%; 'Here is' 2%; 'Sure here's' 2%

(c) Openers. Families have recognisable ways of starting (Table 11). 10% of GPT-4o answers begin with 'Certainly' (next highest family: 5%), while answers opening with 'Here…' are most typical of Llama (27%) and Claude (17%). Because the first tokens are always inside the 256-token window, these cues are available to both networks; the 'drop first & last sentence' ablation below measures how much they matter.

(d) Ablations: removing a signal and retraining. We retrain both networks from scratch on transformed text (train, validation and test transformed alike), so a drop in accuracy measures how much the removed signal was needed and could not be replaced by others:

- *drop first & last sentence*: removes greetings like 'Certainly!' and sign-offs like 'Let me know if…';
- *strip markdown + line breaks*: removes headers, bullets, numbering, bold/italics, code fences and all layout;
- *words only*: lower-cased words, no punctuation, no layout;
- *content words only*: additionally removes all ~320 function words (the classic stylometric signal);
- *style skeleton*: the opposite — every content word becomes `W` (numbers `D`), keeping function words, punctuation and layout, so topic and vocabulary are gone;
- *first 64 tokens*: only the opening of the answer (also removes nearly all length information).

Table 12. RQ4 ablations (test accuracy; CNN: mean ± std over seeds 0–1 where both were run, otherwise seed 0; BiLSTM: seed 0).

Text given to the model	TextCNN acc.	Δ vs full	BiLSTM acc.	Δ vs full
Original response (RQ1)	0.577 ± 0.012		0.576 ± 0.011	
Drop first & last sentence	0.511	-0.066	not run	
Strip markdown + line breaks	0.428 ± 0.006	-0.149	0.443	-0.133
Words only (lower-case, no punctuation)	0.396	-0.181	not run	
Content words only	0.240 ± 0.017	-0.338	0.309	-0.267
Style skeleton (content words → W)	0.589 ± 0.012	+0.011	0.601	+0.025
First 64 tokens only	0.537 ± 0.012	-0.041	0.516	-0.060

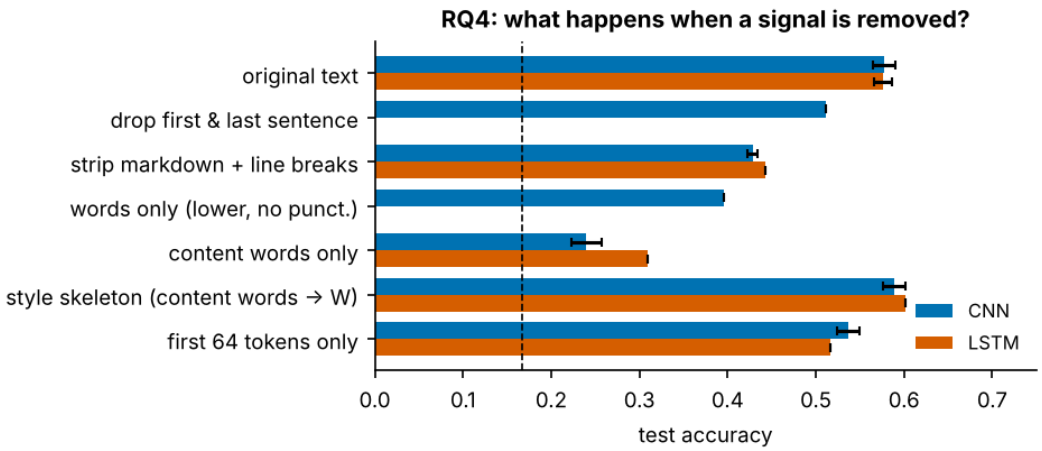


Figure 9. Ablation results (dashed line = chance).

Formatting matters most. Stripping markdown and line breaks lowers CNN accuracy from 0.577 to 0.428 (-0.149) and BiLSTM accuracy to 0.443 (-0.133); among the transformations that remove a single kind of signal, nothing hurts as much as removing layout (compare -0.066 for dropping the first and last sentence and -0.041 for keeping only 64 tokens). Removing punctuation and case as well (words only) lowers it further to 0.396. **Content is not needed.** Strikingly, replacing every content word by `W` does not hurt the CNN (0.589 vs 0.577); the BiLSTM gets 0.601. The networks do not need to know what an answer is about: function words, punctuation and layout are enough, and with only 3,372 training answers the content words seem to act partly as noise that invites over-fitting to topics. **Vocabulary alone is weak.** Content words only (no function words, punctuation or layout) give 0.240 (BiLSTM 0.309), only 0.073 above chance: what an answer talks about says little about who wrote it, because all six models answered the same prompts and therefore cover the same topics. **Openers and sign-offs matter.** Removing only the first and last sentence (everything else kept verbatim) costs 0.066 (0.577 → 0.511): stock phrases such as 'Certainly!', 'What a great question!' or 'I hope this helps' are a real part of the fingerprint, although most of the signal survives without them. Seeing only the first 64 tokens still yields 0.537 (BiLSTM 0.516): the opening of an answer is highly characteristic, and the rest of the response and its length add 0.041.

Answer to RQ4. The classifiers use both *structural* signals (markdown headers, bullets, bold, numbering, line breaks, response length) and *linguistic* signals (openers, function-word and punctuation habits, preferred vocabulary, even typography and language slips). Ranked by the accuracy change when they are removed (the removals are nested: 'words only' also removes layout), the most important are: function words, punctuation and layout (-0.338); punctuation, case and layout (-0.181); formatting/layout (-0.149); opening and closing sentence (-0.066); everything after the first 64 tokens (-0.041). The signals are largely *stylistic rather than topical*: a model that sees no content words at all is as good as one that sees everything. They are also redundant — every ablated model stays above chance — which is why simple countermeasures such as stripping markdown would weaken but not hide an LLM's identity.

8. Lessons and experience

- **Split by prompt, not by row.** With six answers per prompt, a random row split would put the same prompt in train and test and let a model match topics instead of styles. Grouping by prompt is the only way to measure generalisation to new questions.
- **The data design decides what a result means.** The parallel design made RQ2's input-only result exactly 1/6, and the confound experiment showed how easily an unbalanced collection would have produced a convincing but meaningless 'prompt fingerprint'. We learned to ask 'could the label be predicted from how the data were collected?' before believing any number.
- **Check the data, not just the code.** An assertion caught that one file was stored in a different order; without it we would have paired Gemini answers with the wrong prompts. Another check found one empty response, which we

removed for every model to keep classes balanced.

- **Strong simple baselines are humbling and useful.** TF-IDF + logistic regression and even 29 hand-crafted features are about as good as the CNN and LSTM here. With a few thousand examples, inductive bias and features matter more than depth; the neural models' advantage would likely grow with more data or pretrained embeddings.
- **CNN vs LSTM in practice.** One CNN epoch took 17 s versus 35 s for the BiLSTM on one CPU core, and the CNN was about as accurate (0.577 vs 0.576), which fits the finding that the signal is mostly local (n-grams, openers, markdown tokens) rather than long-range. The LSTM needed gradient clipping, packed sequences and a 256-token cap to be affordable.
- **Early stopping and seeds.** Both networks over-fit within a few epochs; restoring the best validation epoch and reporting the mean and spread over three seeds made the comparisons trustworthy.
- **Ablations answer 'why' better than accuracy, but must change only one thing.** Retraining on transformed text turned vague impressions ('GPT uses a lot of markdown') into measured effects. Our first version of the 'drop first & last sentence' transformation re-joined the remaining sentences with line breaks, which silently changed the layout as well and exaggerated the effect (CNN 0.462); after fixing it to keep the middle of the answer verbatim the accuracy was 0.511. An ablation that touches two signals at once answers neither question.

Limitations. One model per family (plus four siblings) cannot represent everything a family produces; outputs come from the decoding settings AlpacaEval used; the task-domain labels come from keyword rules; and the RQ3/RQ4 conditions use one or two seeds, so differences below about 0.02–0.03 should not be over-interpreted. All outputs answer English instructions from 2023-era prompt sets.

Use of AI tools. This project was developed with the assistance of an LLM (Claude, by Anthropic), which the project guidelines allow for understanding concepts, debugging code, brainstorming experiments and interpreting results. Every number in this report is produced by the code in the repository (results/summary.json), so it can be re-checked by re-running the pipeline.

References. Y. Kim, Convolutional Neural Networks for Sentence Classification, EMNLP 2014. S. Hochreiter & J. Schmidhuber, Long Short-Term Memory, Neural Computation 1997. X. Li et al., AlpacaEval: An Automatic Evaluator of Instruction-following Models, github.com/tatsu-lab/alpaca_eval, 2023. Y. Dubois et al., Length-Controlled AlpacaEval, 2024. F. Pedregosa et al., Scikit-learn, JMLR 2011.